

SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY
MUKKA, MANGALURU – 574146



**MICROSERVICES ARCHITECTURE &
IMPLEMENTAION**

MINIPROJECT REPORT

ON

Real Time Chat Application

Submitted in the partial fulfillment of the requirements for the award of the degree of

**BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING**

Submitted By,

**MAHMAD YUNUS
RANJIT DEVADIGA**

**01SU23CS098
01SU23CS148**

Under the guidance of

Mrs. Yogesh
Dept. of IBM

2025-2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SRINIVAS UNIVERSITY, MUKKA

SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY
MUKKA, MANGALURU – 574146



Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Miniproject entitled ***“Real Time Chat Application”*** is a bonafide work carried out by ***Mahmad Yunus, Ranjith Devadiga*** bearing the USN ***01SU23CS098, 01SU23CS148*** in the partial fulfillment for the award of **Bachelor of Technology in Computer Science and Engineering** of the Srinivas University, Institute of Engineering & Technology during the year **2025-2026**. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report deposited in the department library. The Miniproject report has been approved as it satisfies the academic requirements in respect of practical work prescribed for the said degree.

Mrs. Yogesh

Dr. Shifana Begum
(HOD,CSE)

SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY
MUKKA, MANGALURU – 574146



Department of Computer Science and Engineering

DECLARATION

We, **Mahamad Yunus , Ranjith Devadiga** the students of Sixth semester, **B.Tech** in Computer Science and Engineering, Srinivas University, Mukka, hereby declare that the Miniproject entitled ***“Real Time Chat Application”*** has been successfully completed by us in partial fulfillment of the requirements for the award of degree in **Bachelor of Technology in Computer Science and Engineering of Srinivas University, Institute of Engineering and Technology** and no part of it has been submitted for the award of degree or diploma in any university or institution previously.

Date: 24/03/2026

Place: Mukka

Technical Report: Real-Time Chat Application (Microservices Architecture)

Chapter 1: Introduction

1.1 Project Overview

The Real-Time Chat Application is a sophisticated, full-stack communication platform designed to facilitate instantaneous messaging between users within a distributed system. Built using a Microservices Architecture, the application decouples core functionalities into independent, scalable services. This design choice ensures high availability, easier maintenance, and the ability to scale specific components based on demand.

The application is structured to support thousands of concurrent users by distributing load across independent service instances. Each microservice owns its own data store and communicates with other services exclusively through well-defined APIs, eliminating tight coupling. This architectural independence allows the development team to deploy updates to individual services without downtime or risk to the overall system.

The project demonstrates industry-level software engineering practices including containerization with Docker, reverse-proxy routing through an API Gateway, and event-driven communication via WebSocket technology. It serves as a reference implementation for teams looking to migrate from monolithic applications to a scalable microservices paradigm.

1.2 Purpose and Goals

The primary objective of this project is to provide a robust environment for real-time interaction. Key goals include:

- **Scalability:** Utilizing microservices to allow independent scaling of the chat, user, and notification components.
- **Real-Time Performance:** Ensuring minimal latency in message delivery through WebSocket technology.
- **Security:** Implementing secure authentication and authorization mechanisms using

industry-standard protocols.

- **User Experience:** Providing a responsive and intuitive interface for seamless communication.

1.3 Scope of the Project

The scope of this project encompasses the design, development, testing, and deployment of a fully functional real-time chat application. The system supports multi-room chat, user authentication, and live message broadcasting. The following modules are within scope:

Backend Microservices: Four core services — User, Chat, Room, and Notification — each running as an independent Node.js process.

API Gateway: A single entry point that performs request routing, load distribution, and basic rate limiting.

Frontend SPA: A React application offering a complete chat UI including login, room selection, and real-time message display.

Infrastructure: Docker-based containerization enabling one-command deployment on any host machine.

1.4 Problem Statement

Traditional monolithic chat applications suffer from a critical limitation: a single component failure can bring down the entire application. As user bases grow, scaling a monolith requires duplicating the entire application stack, which is expensive and wasteful. Furthermore, deploying a minor bug fix to one feature risks destabilizing unrelated parts of the system.

This project addresses these problems by decomposing the application into discrete services. A failure in the Notification Service, for example, does not impact a user's ability to send or receive messages in real time. Similarly, the Chat Service can be independently scaled by spinning up additional container instances behind a load balancer when user traffic spikes.

Chapter 2: System Architecture

2.1 Architectural Pattern

The system follows a Microservices Architecture pattern, where the frontend communicates with various backend services through a centralized API Gateway. This pattern abstracts the internal complexity of the microservices from the client.

The API Gateway is central to this architecture. It accepts all inbound traffic from the React frontend and routes each request to the appropriate downstream service based on the request path prefix. For example, requests to `/api/users` are forwarded to the User Service, while requests to `/api/rooms` are forwarded to the Room Service. This single-entry-point design simplifies client code and allows security policies such as JWT validation to be enforced in one place.

2.2 Component Overview

The architecture consists of the following key components:

Component	Description
Frontend	A React-based single-page application (SPA) providing the user interface.
API Gateway	Acts as the entry point, routing requests to the appropriate microservices.
User Service	Manages user identity, authentication (JWT), and profile data.
Chat Service	Handles real-time message broadcasting and message persistence.
Room Service	Manages the creation, joining, and listing of chat rooms.
Notification Service	Sends automated email notifications to users.

2.3 Service Communication

The system employs two primary communication methods:

- 1 Synchronous (REST): Used for standard CRUD operations such as user registration, room creation, and profile updates.
- 2 Asynchronous (WebSockets): Facilitated by Socket.IO for real-time events like message delivery and typing indicators.

2.4 Data Flow Diagram

The following describes the step-by-step data flow for a typical message send operation:

Step 1 — Authentication: The user logs in via the React frontend. The API Gateway forwards credentials to the User Service, which validates them and returns a signed JWT.

Step 2 — Room Selection: The authenticated user selects or creates a chat room. The frontend calls the API Gateway, which routes the request to the Room Service. The Room Service returns room metadata and the list of current members.

Step 3 — WebSocket Connection: The frontend establishes a WebSocket connection directly to the Chat Service, passing the JWT as a query parameter for authentication.

Step 4 — Message Transmission: When the user sends a message, the frontend emits a `sendMessage` event over the WebSocket. The Chat Service persists the message in MongoDB and broadcasts a message event to all sockets subscribed to that room.

Step 5 — Notification Trigger: For certain events such as new user registration, the User Service calls the Notification Service via an internal HTTP request to trigger a welcome email.

2.5 Deployment Architecture

All services are containerized using Docker. A `docker-compose.yml` file defines the full application stack, including one container per service, a shared MongoDB container, and a network bridge enabling inter-container communication. This approach guarantees environment consistency across developer machines, CI/CD pipelines, and production servers.

Each service exposes a specific port internally; the API Gateway is the only service with a port mapped to the host machine. This network isolation improves security by preventing direct external access to individual backend services.

Chapter 3: Technology Stack

3.1 Frontend Technologies

- React & Vite: Used for building a fast, component-based user interface.
- Socket.IO Client: Enables real-time, bi-directional communication with the Chat Service.
- Axios: Handles HTTP requests to the API Gateway.
- Tailwind CSS: Provides a responsive and modern design framework.

3.2 Backend Technologies

- Node.js & Express: The runtime and framework used for developing all microservices.
- Socket.IO: Powers the real-time engine for the Chat Service.
- JSON Web Tokens (JWT): Used for secure, stateless authentication across services.
- Nodemailer: Integrated into the Notification Service for sending emails.

3.3 Database and Infrastructure

- MongoDB: A NoSQL database used for storing users, messages, and room data.
- Mongoose: An ODM (Object Data Modeling) library for MongoDB and Node.js.
- Docker & Docker Compose: Used for containerization and orchestration, ensuring consistent environments across development and production.

3.4 Development Tools and Environment

The development workflow was supported by a set of modern tooling choices designed to maximize productivity and code quality:

VS Code: The primary IDE used across all services, with extensions for ESLint, Prettier, and Docker management.

Postman: Used for testing REST API endpoints during development of each microservice.

Git & GitHub: Version control was managed using Git with a feature-branch workflow, and the repository was hosted on GitHub for collaboration and code review.

ESLint & Prettier: Enforced consistent code style and caught common JavaScript errors at development time.

Nodemon: Used in development mode for each backend service to automatically restart the server process when source files change, reducing development iteration time.

3.5 Rationale for Technology Choices

Node.js was selected as the backend runtime for its event-driven, non-blocking I/O model, which is particularly well-suited to I/O-intensive workloads like real-time messaging. Unlike traditional multi-threaded servers that allocate a thread per connection, Node.js handles concurrent connections through a single event loop, enabling it to support large numbers of simultaneous WebSocket connections with minimal memory overhead.

MongoDB was chosen as the data store due to its flexible schema design, which allows message and user documents to evolve over time without requiring complex database migrations. Its native support for JSON-like documents aligns naturally with the JavaScript/Node.js stack, and Mongoose provides a clean abstraction layer for defining and validating data models.

React was selected for the frontend for its component-based architecture, rich ecosystem, and wide industry adoption. The use of Vite as the build tool significantly reduces development server startup time compared to traditional Create React App, enabling faster iteration cycles.

Chapter 4: Microservices Details

4.1 User Service

The User Service is responsible for the entire lifecycle of a user's account. It implements:

- Registration & Login: Secure password hashing using bcryptjs.
- Authentication: Issuing JWTs upon successful login, which are then used to authorize requests to other services.
- Profile Management: Allowing users to view and update their personal information and status (online/offline).

4.2 Chat Service

As the core of the application, the Chat Service manages real-time interactions. It utilizes Socket.IO to maintain persistent connections. When a user sends a message, the service:

- 3 Persists the message to the MongoDB database.
- 4 Broadcasts the message to all connected clients in the specific room.
- 5 Handles auxiliary events such as "typing" and "stopTyping" indicators.

The Chat Service maintains a mapping of Socket IDs to user identities and room memberships.

When a user disconnects unexpectedly, a disconnect event listener cleans up their presence and notifies other users in the room. This ensures that the typing indicator and online presence list remain accurate even when connections drop unexpectedly.

Messages are stored in a MongoDB collection with the following schema fields: message content, sender ID, room ID, and a UTC timestamp. When a user joins a room, the Chat Service fetches the last N messages from the database and emits them to the joining client, populating the message history immediately on connection.

4.3 Room Service

The Room Service handles the logic for grouping users. Features include:

- **Room CRUD:** Creating, updating, and deleting chat rooms.
- **Membership Management:** Logic for users joining or leaving specific rooms, maintaining a list of active members for each room.

Each room document stored in MongoDB contains a name, a description, a creator reference, a creation timestamp, and an array of member user IDs. The Room Service exposes REST endpoints for listing all available rooms, fetching details of a specific room, creating a new room, and updating room metadata. All write operations require a valid JWT, preventing anonymous users from creating or modifying rooms.

Room deletion is restricted to the original creator of the room. When a room is deleted, the Room Service publishes an internal event that the Chat Service listens for, prompting it to close all active WebSocket subscriptions to that room and notify connected clients that the room is no longer available.

4.4 Notification Service

This service operates independently to handle outbound communications. It is triggered by other services (e.g., the User Service after registration) to send:

- **Welcome Emails:** Sent to new users upon successful registration.
- **System Alerts:** Notifications regarding account changes or important updates.

The Notification Service is implemented as a stateless HTTP microservice. It exposes a single internal endpoint that accepts a payload specifying the recipient email address, subject line, and message body. The service uses Nodemailer configured with an SMTP transport to relay the email through a mail server. Environment variables are used to configure the SMTP credentials, ensuring no sensitive data is hardcoded in the source code.

Chapter 5: Implementation Details

5.1 Real-Time Workflow

The real-time messaging flow is implemented as follows:

- 6 Connection: The client establishes a WebSocket connection with the Chat Service.
- 7 Joining: The client emits a `joinRoom` event with the target `roomId`.
- 8 Messaging: When a message is sent, a `sendMessage` event is emitted. The server saves this to the `messages` collection and emits a `message` event to all sockets in that room.

5.3 API Gateway Configuration

The API Gateway is built using `http-proxy-middleware` on top of an Express server. Route matching is configured declaratively: each route prefix maps to a target service URL and port. The gateway also performs JWT validation for protected routes by inspecting the Authorization header and verifying the token signature using the shared JWT secret. Requests with invalid or missing tokens are rejected at the gateway with a 401 Unauthorized response before they reach any downstream service.

The gateway also handles CORS configuration globally, setting the appropriate response headers to allow requests from the frontend origin. This centralizes CORS policy management, eliminating the need to configure it independently in each microservice.

5.4 Error Handling Strategy

Each microservice implements a consistent error handling pattern using Express middleware. All route handlers are wrapped in try-catch blocks; unhandled errors are forwarded to a centralized error handler that formats the error response as a JSON object containing a status code and error message. This ensures that all API errors are returned in a predictable format that the frontend can reliably parse and display.

For WebSocket events, error handling is implemented using Socket.IO's built-in error event.

The Chat Service emits an error event to the specific client socket when an operation fails, allowing the frontend to display appropriate feedback without crashing the socket connection.

5.5 Database Schema Design

Three primary Mongoose schemas are defined across the microservices:

User Schema: Fields include username (unique, required), email (unique, required), password (hashed), status (online/offline), and createdAt timestamp. The User Service uses this schema exclusively.

Room Schema: Fields include name (unique, required), description, createdBy (reference to User), members (array of User references), and createdAt timestamp. Used by the Room Service.

Message Schema: Fields include content (required), sender (reference to User), room (reference to Room), and createdAt timestamp. Indexed on the room field for efficient retrieval of room message history. Used by the Chat Service.

Each service connects to the same MongoDB instance but operates exclusively on its own collections. This is a pragmatic simplification for a development-stage application; in a production system, each service would ideally maintain its own isolated database to fully enforce the microservices principle of data ownership.

5.2 Security Implementation

Security is handled at multiple levels:

- Data Encryption: Passwords are never stored in plaintext; they are hashed using bcrypt.
- Stateless Auth: The API Gateway and individual services verify the JWT in the Authorization header for protected routes.
- CORS: Configured to allow requests only from trusted origins (the frontend and gateway).

5.6 Testing Approach

The application was tested at multiple levels to ensure correctness and reliability:

Unit Testing: Individual functions such as password hashing, JWT generation, and Mongoose model validation were tested in isolation using Jest.

Integration Testing: Each REST API endpoint was tested end-to-end using Postman collections, covering success cases, authentication errors, and validation errors.

Manual WebSocket Testing: The real-time functionality was tested manually by opening multiple browser tabs simulating different users and verifying message delivery, typing indicators, and room join/leave behaviour.

Load Testing: Basic load tests were conducted using Artillery to simulate 100 concurrent WebSocket connections and verify that message delivery remained stable under load.

Chapter 6: Features and Functionalities

6.1 Core Features

- Real-Time Messaging: Instant delivery of text messages across rooms.
- Room Management: Users can create public rooms and browse existing ones.
- Authentication: Secure login/logout and persistent sessions via local storage and JWT.
- Typing Indicators: Visual feedback when other users are composing a message.

Online Presence: The sidebar displays which users are currently online within a room, updated in real time as users connect and disconnect.

User Profiles: Each user has a profile page showing their display name, email, and current online status.

Notification Emails: Automated welcome emails are sent to new users upon registration, confirming their account details.

6.2 UI/UX Enhancements

- Responsive Design: The interface adapts to various screen sizes (mobile, tablet, desktop).
- Dark Mode: A modern, dark-themed UI for reduced eye strain.
- Message History: Automatic loading of previous messages when joining a room.

Smooth Animations: Transitions and hover effects are applied to interactive elements, providing visual feedback that improves perceived performance.

Error States: Invalid form submissions and network errors are surfaced with clear, user-friendly error messages rather than silent failures.

Loading Indicators: Spinner overlays are shown during API calls to inform the user that an operation is in progress.

6.3 Accessibility Considerations

Accessibility was considered during the design of the user interface. All interactive elements

include appropriate ARIA labels, ensuring compatibility with screen readers. Sufficient colour contrast ratios are maintained throughout the dark-mode theme to meet WCAG 2.1 AA standards. Keyboard navigation is fully supported across the main chat interface, allowing users to navigate between rooms, compose messages, and send them without requiring a mouse.

6.4 Performance Optimisations

Several frontend performance optimisations were implemented to ensure a smooth experience even on low-bandwidth connections:

Code Splitting: React lazy loading is used to split the bundle by route, ensuring that only the code needed for the current view is loaded initially.

Debounced Typing Events: The typing indicator emits events to the server only after a short debounce delay, reducing unnecessary WebSocket traffic when a user types quickly.

Pagination of Message History: Rather than loading the entire message history for a room on join, messages are fetched in pages of 50, with older messages loaded on demand as the user scrolls upward.

Memoization: React.memo and useMemo hooks are applied to computationally expensive components and derived values to prevent unnecessary re-renders.

Chapter 7: Conclusion

7.1 Summary

The Real-Time Chat Application demonstrates a successful implementation of a modern microservices-based web application. By leveraging Node.js, Socket.IO, and React, the project achieves high performance and scalability while maintaining a clean separation of concerns.

The project has provided deep practical experience with the challenges of distributed systems, including inter-service communication, shared authentication state, and ensuring data consistency across independently deployed services. The Docker-based deployment model proved invaluable for maintaining environment consistency and simplifying the development setup process for new team members.

The use of WebSockets for real-time communication demonstrated measurably lower latency compared to HTTP polling in informal benchmarks, confirming that the architectural choice was appropriate for the problem domain. The decoupled nature of the Notification Service, in particular, illustrated the resilience benefits of microservices: when the email SMTP server was temporarily unavailable during testing, core chat functionality continued uninterrupted.

7.3 Lessons Learned

Several valuable lessons were drawn from the development process:

Service Boundary Definition: Defining clear, stable boundaries between services at the outset is critical. Early in the project, the Chat and Room services were initially conceived as a single service. Separating them mid-development required careful refactoring and highlighted the importance of thorough up-front design.

Distributed Debugging: Debugging issues that span multiple services is significantly more complex than debugging a monolith. Investing in structured logging across all services early in the project would have accelerated diagnosis of inter-service communication issues.

Token Propagation: Ensuring that the JWT is correctly propagated from the API Gateway

through to individual services required careful coordination. A shared JWT verification middleware module, imported as a local package by each service, proved to be an effective solution.

WebSocket Scalability: Running multiple instances of the Chat Service introduces a challenge: a message received by one instance must be broadcast to clients connected to other instances. In a production system, this is solved using a message broker such as Redis Pub/Sub. Implementing a basic Redis adapter for Socket.IO was explored as a stretch goal and demonstrated the pathway to horizontal scaling.

7.2 Future Enhancements

Potential areas for future development include:

- **Private Messaging:** Implementing one-on-one direct messaging between users.
- **File Sharing:** Adding support for sending images, videos, and documents.
- **Message Encryption:** Implementing end-to-end encryption for enhanced privacy.
- **Push Notifications:** Integrating mobile push notifications for real-time alerts on mobile device
- **Voice and Video Calls:** Integrating WebRTC-based peer-to-peer audio and video calling directly within chat rooms.
- **Message Search:** Full-text search across message history using Elasticsearch or MongoDB Atlas Search.
- **Role-Based Access Control:** Introducing room administrator roles with permissions to moderate messages and manage membership.
- **Microservices Observability:** Integrating distributed tracing (e.g., OpenTelemetry with Jaeger) and centralised log aggregation (e.g., ELK Stack) for production-grade monitoring and debugging.
- **Kubernetes Deployment:** Migrating the Docker Compose deployment to a Kubernetes cluster to support auto-scaling, rolling updates, and self-healing infrastructure.

- Voice and Video Calls: Integrating WebRTC-based peer-to-peer audio and video calling directly within chat rooms.
- Message Search: Full-text search across message history using Elasticsearch or MongoDB Atlas Search.
- Role-Based Access Control: Introducing room administrator roles with permissions to moderate messages and manage membership.
- Microservices Observability: Integrating distributed tracing (e.g., OpenTelemetry with Jaeger) and centralised log aggregation (e.g., ELK Stack) for production-grade monitoring and debugging.
- Kubernetes Deployment: Migrating the Docker Compose deployment to a Kubernetes cluster to support auto-scaling, rolling updates, and self-healing infrastructure.